Comparing two hashes is somewhat similar to sets, the Hash class defines > as true if the left side is a proper subset of the right, meaning that every key value pair on the left side must be on the right side, and the right side has more pairs. The >= method is a regular subset method, so the two hashes can be equal. The < and <= methods work for proper superset and superset.

Hashes do not implement their own implementation of <=>, taking the parent class (Object) implementation. This appears to return true if the hashes are equal and nil otherwise, which is to say that it's probably not a useful way to compare Hashes.

Modifying Hashes

There are several methods that work to modify hashes. The clear method removes all key/value pairs from the hash, while compact removes all pairs where the value is nil.

Hashes respond to the same filter, select, and reject methods as other classes that include Enumerable, but as with each, the block takes two arguments: the key and the value. Hashes also define filter!, select!, and reject! bang, which modify the original argument in place rather than returning a new hash.

Hashes also respond to shift, which removes and returns the first key/value pair.

If you want a subset of a hash, you can use the slice method, which takes an arbitrary number of arguments and returns a new hash containing only key/value pairs whose keys are in the list of arguments.

Converting Hashes

Hashes can be converted or transformed in a variety of ways. The to_a method converts a hash to an array of two-element [key, value] arrays. If you want a single-dimensional array, you can use flatten, which is hash.to_a.flatten. By default, flatten does not flatten values that happen to be arrays—you can do that with an optional argument that specifies how many levels of sub-array values are flattened. The default is 1, and a negative argument flattens all values no matter how deeply nested.

The to_s method converts to a string, and the only reason to mention it here is that the string uses key => value syntax no matter which syntax was used to create the hash.

The invert method creates a new hash where all values are keys and all keys are values. If values are shared across multiple keys, keys that are later in the hash sequence will replace earlier keys.

A fun conversation is to_proc. Hash#to_proc returns a Proc that takes arguments and uses those arguments as though they were keys to the hash, returning the associated value. A side effect of the existence of to_proc means that the same & trick that we often use for symbols can also be used for hashes:

```
hash = {a: 1, b: 2, c: 3}
hash.to_a               # => [[:a, 1], [:b, 2], [:c, 3]]
hash.flatten            # => [:a, 1, :b, 2, :c, 3]
hash.to_s               # => "{:a=>1, :b=>2, :c=>3}"
hash.invert             # => {1=>:a, 2=>:b, 3=>:c}
hash_proc = hash.to_proc
hash_proc.call(:a)      # => 1
[:a, :b, :c].map(&hash) # => [1, 2, 3]
```